Short Circuit

System Design

Designing Data-Intensive Applications
by Martin Kleppmann & Chris Riccomini
Second Edition
Up until Chapter 4: Storage and Retrieval

Summary by Emiel Bos

1 Introduction

While in *compute-intensive systems* the challenge is parallelizing a very large computation, in *data-intensive applications* data management is one of the primary challenges. Historically, only large organizations had the large data volumes that required sophisticated technical solutions, but now it's common for smaller companies and startups to build data-intensive systems. For web applications and mobile apps, the client-side code (which runs in a web browser or on a phone) is called the *frontend*, and the server-side code that handles user requests is known as the *backend*. A frontend needs to handle only one user's data (often managed locally on the user's device), whereas the backend manages data on behalf of all the users, so the greatest data infrastructure challenges commonly lie in the backend. Application code is often *stateless*; when it finishes handling one HTTP request, it forgets everything about that request. Any information that needs to persist between requests needs to be stored either on the client or in the server-side *data infrastructure* (i.e. databases and any additional data systems, such as caches or message queues).

A database *transaction*[1] refers to a low-latency group of reads and writes that form a logical unit.

Since the 2010s, the roles of *software development* (software engineers who develop the software) and *operations* (database/system administrators who configure infrastructure, deploy application, and monitoring and diagnosing any problems) merged into teams with a shared responsibility for both backend services and data infrastructure; the *DevOps* philosophy. *Site reliability engineers* (SREs) are Google's implementation of this. The DevOps/SRE philosophy prefers repeatable automation over manual one-off jobs, using ephemeral VMs and services over long-running servers, enabling frequent application updates, learning from incidents, and preserving the organization's knowledge about the system as individual people come and go.
The role of *observability* involves collecting data about the execution of a system and allowing that data to be queried in ways that allow both high-level metrics and individual events (with tracing tools) to be analyzed.

2 Trade-offs

Several contrasting concepts and their trade-offs.

2.1 Operational systems vs. analytical systems

There is a distinction between two types of systems that manage different types of data for different audiences:

- *Operational systems* consist of the backend services and data infrastructure where data is created (e.g. by serving external users). Application code both reads and modifies the data in its databases. Reads are typically for an individual records using a key (called *point queries*). Writes (insertions, updates, or deletions) are based on the

[1] This term originated in the early days of business data processing, when a write to the database typically corresponded to a commercial transaction taking place (a sale, an order, a salary payout, etc.). Now, databases don't only involve money changing hands, but the basic access pattern remains similar to processing business transactions.

user's input. Their access pattern became known as *online transaction processing* (OLTP) and is characterized by lots of small, fixed-in-code queries in a database with gigabytes to terabytes of the latest, up-to-date data. A large enterprise may have many, often independent OLTP systems.

- *Analytical systems* contain a read-only copy of the data from the operational systems and are optimized for data processing for analytics/business intelligence. They have a very different access pattern than OLTP, usually scanning over a huge number of records and calculating aggregate statistics rather than returning individual records. Their access pattern is known as *online analytical processing* (OLAP) and is characterized by few, complex queries in a database with terabytes to petabytes of the historical data. A large enterprise generally has one OLAP system, which can consist of one or two types:
  - *Data warehouse*; a separate, often relational/SQL database system that contains a read-only copy of the data from all the various OLTP systems and that analysts can query to their hearts' content, without affecting OLTP operations and performance. Data warehouses often store data very differently from OLTP databases. They're populated with a process called *extract–transform–load* (ETL), where data is extracted from OLTP database[2], transformed into an analysis-friendly schema, cleaned up, and loaded into the data warehouse (or ELT, if the order of the transform and load steps is swapped and the transformation is done in the data warehouse after loading). In some cases the outputs of analytical systems are made available to operational systems (*reverse ETL*). ETL processes have been generalized to *data pipelines*.
  - *Data lake*; similar to a data warehouse, but a data lake simply contains files (encoded database records, text, images, videos, sparse matrices, feature vectors, etc.) without imposing any particular file format, data model, or schema. Besides being more flexible, a data lake is also often cheaper than relational data storage. Data lakes store data in the raw form produced by the operational systems, and can therefore be used as an intermediate stop on the path from the operational systems to the data warehouse (i.e. without the transformation into a relational data warehouse schema).

Several job functions work with data: *backend engineers* build the operational systems that handle requests for reading and updating data, *business analysts* generate business intelligence reports about the activities of the organization for the management using analytical systems (often data warehouses), *data scientists* look for novel insights in data or create features enabled by data analysis/machine learning with analytical systems (often data lakes), *data engineers* integrate the operational and analytical systems and take responsibility for the wider data infrastructure, and *analytics engineers* model and transform data to make it more useful for the business analysts and data scientists.

Some database systems offer *hybrid transactional/analytical processing* (HTAP) for OLTP and analytics in a single system without requiring ETL. However, many HTAP systems internally consist of an OLTP system coupled with a separate analytical system, hidden behind a common interface—so the distinction between the two remains important for understanding how these systems work.

A related distinction of systems is that between a *system of record* aka a *source of truth*, which holds the authoritative/canonical version of data and to which new data (e.g. user input) is first written, and a *derived data system*, containing transformed/processed data from another system (e.g. an index or cache). This data is *redundant* (it duplicates existing information), but often essential for getting good performance on read queries or for obtaining another perspective of the data. Analytical systems are usually derived data systems.

2.2 Self-hosted systems vs. cloud services

The spectrum of decisions on outsourcing software and its operations can be roughly partitioned into the following points, from more control and greater investment to less control and lower investment[3]:

- In-house software, in-house operations; e.g. bespoke application code. As a rule of thumb, things that are a core competency or a competitive advantage of your organization should be done in-house. In-house operations means you control the server, which can either be *on-premises* (your own hardware or rented servers) or *infrastructure-as-a-service* (IaaS) (cloud infrastructure).
- In-house software, outsources operations; e.g. *serverless* aka *function-as-a-service* (FaaS). The cloud provider automatically scales hardware resources to match demand of your service.
- Off-the-shelf software, in-house operations; e.g. self-hosted PostgreSQL on IaaS. Off-the-shelf software can also

[2]External SaaS products may also serve as data sources. In those cases, the data is accessible only via the software vendor's API, but bringing the data into your own data warehouse can enable more analyses. ETL for SaaS APIs is often implemented by specialist data connector services such as Fivetran, Singer, or Airbyte.

[3]There are more points along this spectrum, such as taking open source software and running a modified version of it.

mean open source.

- Off-the-shelf software, outsourced operations; e.g. cloud services/SaaS that you access only through a web interface or API. As a rule of thumb, things that are non-core, routine, or commonplace should be left to a vendor. Whether using a cloud service is actually cheaper and easier than self-hosting depends very much on your skills and the workload on your systems. Cloud services are particularly valuable if the load on your systems varies a lot over time (this is often the case for analytical systems), because else you'd have to provision for peak load and have most of your resources idle during quiet periods. The biggest downside of a cloud service is that you have no control over it. On a technical level, the term *cloud native* is used to describe an architecture that is designed to take advantage of cloud services. Systems that have been designed from the ground up to be cloud native (e.g. Aurora) have been shown to have several advantages over self-hosted software (e.g. PostgreSQL): better performance, faster recovery, quick scaling, and larger datasets. Cloud native services build upon lower-level cloud services (e.g. S3) to create higher-level services (e.g. Snowflake, a cloud-based analytical data warehouse that relies on S3). In the cloud, compute instances (VMs) may have local disks attached, but they're typically treated as ephemeral cache and less like long-term storage. Cloud services do offer virtual disk storage that isn't tied to one instance and that emulates the behavior of a disk (a *block device*, where each block is typically 4 KiB in size) (e.g. Amazon EBS), but the emulation introduces overheads avoided by systems designed for the cloud. Therefore, cloud native services generally rely on dedicated storage services, such as object storage services (e.g. S3), which are designed for long-term storage of (large) files. Cloud native systems are often *multitenant*, which means that data and computation from several customers are handled on the same shared hardware by the same service, for better hardware utilization, easier scalability, and easier management; though it requires careful engineering to avoid performance hampers between clients. While the goal of the role of operations remains the same (providing a reliable service), the processes and tools have evolved in the cloud era; it's now about choosing and managing cloud services, and capacity planning and performance optimization have become financial planning and cost optimization.

High-performance computing (HPC) aka *supercomputing* is another way of building large-scale computing systems. Although there is overlap with cloud computing, supercomputers are typically used for computationally intensive scientific computing tasks (e.g. weather forecasting, climate modeling, molecular dynamics, etc.) rather than serving many user requests with high availability, running large batch jobs that periodically checkpoint the state of their computation to disk, communicate through shared memory and RDMA with high bandwidth and low latency (because they assume a high level of trust between the users of the system) rather than requiring strong security mechanisms, use specialized network topologies (e.g. multidimensional meshes and toruses), and are often geographically close to each other. Large-scale analytical systems sometimes share some characteristics with supercomputing. Apart from that, supercomputing is out-of-scope.

2.3 Single-node systems vs. distributed systems

If a workload can run on a single node (with maybe a single-node databases such as DuckDB, SQLite, and KùzuDB), that is often much simpler and cheaper than setting up a distributed system, but that's not always possible. A *distributed system* is a system that involves several machines called *nodes* communicating via a network. Sometimes your system is inherently distributed, like when you have interacting users on separate devices, or when data is stored in one cloud service but processed in another. A distributed system offers *fault tolerance/high availability* (there are redundant machines in case one goes down), *scalability* (spread load too large for one machine across multiple machines), lower *latency* (servers are geographically distributed to reduce the travel distance to customers), *elasticity* (deployment scales up or down according to demand), different types of (specialized) hardware for different workloads, legal compliance (servers in different countries store and process different data according to data residency law), and *sustainability* (the flexibility to run jobs when plenty of renewable electricity is available and not when the power grid is under strain). The downside is that every request and API call needs to deal with the possibility of failure, and is vastly slower than calling a function in the same process. When each service has its own database, maintaining consistency of data across those different services becomes a problem.

The most common way of distributing a system is to divide the machines into clients and servers, with the clients making requests to the servers via HTTP. In a *microservices* architecture, a complex application is decomposed into small, loosely-coupled, and autonomously deployable services that each have one well-defined purpose, API, and team that maintains it. This contrasts with the conventional *monolithic* architecture, where an application is constructed as a single, tightly-coupled unit. Microservices have a single responsibility; can be independently developed, deployed, and scaled; are decentralized, with each service possessing its data and business logic, interact with each other using lightweight protocols (e.g. HTTP/REST, gRPC, or message queues), and are fault tolerance. On the other hand, having many services can itself breed complexity: testing can be complicated, since you also need to run all the dependency services; each service requires infrastructure for deploying, scaling, and observability (though orchestration frameworks such as Kubernetes provide a foundation for this infrastructure); and microservice APIs can be challenging to evolve because clients depent on it.

Microservices are primarily a technical solution to a people problem, namely allowing different teams to make progress independently without having to coordinate with each other, but this is often only a benefit in large organizations.

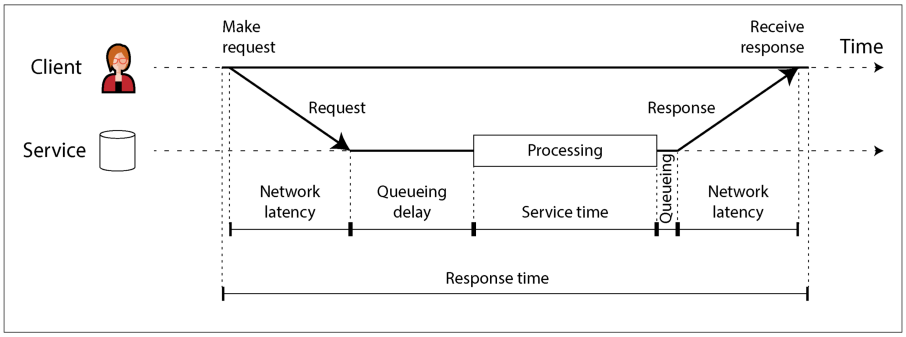
2.4 Business vs. society

Besides the needs of their own business, data systems also have a responsibility toward society at large. Since 2018, the GDPR has given Europeans greater control and legal rights over their personal data, and similar privacy regulations have been adopted in various other countries and states (e.g. the CCPA). Regulations around AI, such as the EU AI Act, place further restrictions on how personal data can be used. Legal considerations are influencing the very foundations of data system design. There aren't clear guidelines on which particular technologies or system architectures should be considered GDPR compliant. In general, we store data because we think that its value is greater than the costs of storing it, but costs should take into account the risks of liability and reputational damage if the data is leaked, compromised by adversaries, or found non-compliant. Governments or police forces might also compel companies to hand over data. The principle of *data minimization* runs counter to the "big data" philosophy of storing lots of data speculatively.
Social media has changed how individuals consume news and form their political opinions. Automated systems increasingly make decisions that have profound consequences for individuals.

3 Nonfunctional requirements

Functional requirements describe the functionality that an application must offer. *Nonfunctional requirements* describe the qualities that a system should have. Many nonfunctional requirements are out-of-scope, such as security.

3.1 Performance



Response time is the elapsed time between a request and the response (the round trip time) in (milli/micro)seconds, and is usually what users care about.[4] *Throughput* is the number of requests (or the data volume, or something else) per second, and determines the required computing resources. For a given allocation of hardware resources, there is a maximum throughput that can be handled. A system is *scalable* if its maximum throughput can be increased proportionally by adding computing resources. Throughput and response time are often related; as the throughput of a service approaches its capacity, the response time increases dramatically, because of *queueing* (incoming requests needs to wait until earlier requests have completed). When throughput pushed close to the limit, systems can sometimes enter a vicious cycle where it becomes less efficient and hence even more overloaded, for example when clients time out and resend their requests (a *retry storm*). If the system remains in an overloaded state until it is rebooted/reset, it's in a *metastable failure*. To avoid a retry storm, you can increase and randomize the time between successive retries on the client side (*exponential backoff*) and temporarily stop sending requests to a service that has returned errors or timeouts recently (by using a *circuit breaker* or *token bucket algorithm*). The server can also detect when it is approaching overload and start proactively rejecting requests (*load shedding*), or send back responses asking clients to slow down (*backpressure*).

Response time includes *service time* (the duration for which the service is actively processing the request), and *latency*[5] (a catchall term for time during which a request is not being actively processed, i.e. during which it is latent). Latency, in turn, includes *network latency* (the time that a request and response spend traveling through the network), and *queueing delays* (can occur at several points in the flow). Because the response time varies between requests, we think of it as a distribution of values. The average response time (technically, the arithmetic mean) is useful for estimating throughput

[4] While a faster service is obviously better, there are surprisingly few reliable studies and data that quantify the effect that latency has on user behavior.

[5] Latency is also sometimes used synonymously with response time.

limits, but it's better to use the median (the 50th percentile). To figure out how bad your outliers are, you can look at e.g. the 95th, 99th, and 99.9th percentiles (known as *tail latencies*).[6] These can be important, e.g. for an e-commerce company, customers with the slowest requests are often those who have the most data on their accounts, and are therefore the most important. On the other hand, reducing response times at very high percentiles is difficult due to random factors outside of your control, and the benefits are diminishing. The chance of getting a high response time user request increases if that request requires multiple backend calls, because it is bottlenecked by the slowest such calls (an effect known as *tail latency amplification*). Percentiles are often used in *service level objectives* (SLOs), which are minimum/expected targets/goals for performance and/or availability. A *service level agreement* (SLA) is a contract states what compensation/refund should be payed if the SLO isn't met.

3.2 Reliability and Fault Tolerance

Reliability roughly means the degree to which a system continues to work correctly even when things go wrong, or, the probability that the system keeps working. "Going wrong" can be due to a *fault* (when a particular part of a system stops working correctly) or a *failure* (when the system as a whole stops providing the required service – it does not meet the SLO); those are the same thing, just at different levels. A system is *fault-tolerant* if it continues providing the service in spite of faults. If a system cannot tolerate a certain part becoming faulty, that part is a *single point of failure* (SPOF). Fault tolerance is always limited to a certain maximum per type of faults, e.g. a maximum of one out of three nodes crashing. In such fault-tolerant systems, it can make sense to increase the rate of faults by triggering them deliberately in order to continually exercise and test the fault tolerance of a system. This is called *fault injection*, and it falls under the discipline of *chaos engineering*, which aims to improve confidence in fault-tolerance mechanism.

Humans design, build, and maintain software systems, and humans make mistakes. Blaming people for mistakes is counterproductive. What we call "human error" is not really the cause of an incident, but rather a symptom of a problem with the sociotechnical system in which people are trying their best to do their jobs. Measures to minimize the impact of human mistakes include thorough testing, rollback mechanisms, gradual rollouts of new code, detailed and clear monitoring, observability tools for diagnosing production issues, and clear user interfaces. However, many organizations choose more features over more testing. Then, when a preventable mistake inevitably occurs, blaming the person who made the mistake does not make sense; the problem is the organization's priorities. Organizations are increasingly doing *blameless postmortems*: in which the people involved in an incident are encouraged to share full details about what happened, without fear of punishment.

3.3 Scalability

Scalability is a system's ability to cope with increased load. In the best case, investments in scalability are wasted effort and premature optimization; in the worst case, they lock you into an inflexible design and make it harder to evolve your application. You discuss growth questions only when you have a clear understanding of the current load on the system, often measured in terms of throughput, but it can also be the ratio of reads to writes in a database, the hit rate on a cache, the number of data items per user, etc. Then, you can investigate the effect when the load increases; either on performance while keeping the system resources unchanged, or on the required system resources to keep performance unchanged. Usually the goal is to keep the performance of the system within the requirements of the SLA while also the cost of running the system. *Linear scalability* means that the load you can handle increases proportionally with resources while keeping performance the same, and it is considered a good thing. Better than linear scalability could be because of economies of scale or a better distribution of peak load, but more likely is that the cost grows faster than linearly.
The architecture of systems that operate at large scale is usually highly specific to the application; there is no such as *magic scaling sauce*. Also, an architecture that is appropriate for one level of load is unlikely to cope with 10 times that load, and you will probably need to rethink your architecture on every order of magnitude load increase. It is usually not worth planning future scaling needs more than one order of magnitude in advance, because the needs of the application are likely to evolve as well. A good general principle for scalability is to break a system into smaller, largely independent components.

Vertical scaling or *scaling up* adds hardware redundancy to existing machines (e.g. more CPUs, more RAM, or more disks). A *shared-memory architecture* is when you have parallelism on a single machine by using multiple processes or threads that can access the same RAM. However, the cost grows faster than linearly, because a high-end machine with twice the resources as a low-spec machine is more than double the price, and is actually also unlikely to be able to handle twice the load because of bottlenecks. The *shared-disk architecture* uses several machines with independent CPUs and

[6] If you want to add response time percentiles to your monitoring dashboards, you could keep a rolling window of response times for requests in the last x minutes, sort that list every minute and calculate based on it. If that is too inefficient, there are algorithms that can calculate a good approximation of percentiles at minimal CPU and memory cost. Note that averaging percentiles is mathematically meaningless; you should aggregate by summing the histograms.

RAM but stores data on a shared array of disks connected via a fast network, such as *network-attached storage* (NAS) or a *storage area network* (SAN). This architecture has traditionally been used for on-premises data warehousing workloads, but contention and the overhead of locking limit the scalability of the shared-disk approach.

Concerning reliability, vertical scaling is most effective when component faults are independent (one fault does not change the likelihood of another fault), but experience has shown correlations between component failures. For this reason, cloud systems tend to focus less on the reliability of individual machines and more on tolerating faulty nodes at the software level, through horizontal scaling.
Horizontal scaling or *scaling out* adds more machines with independent resources (therefore also called a *shared-nothing architecture*) that coordinate over a network at the software level. This has the potential to scale linearly, can use whatever hardware offers the best price/performance ratio (especially in the cloud), can more easily adjust its hardware resources with load variations, and can achieve greater fault tolerance by geographically distributing the system. Cloud providers use *availability zones* to identify which resources are physically co-located, which are more likely to fail at the same time. On the other hands, software faults are often highly correlated, because it is common for many nodes to run the same software and thus have the same bugs. However, distributed systems are inherently complex.
A multi-node fault-tolerant system can be patched by restarting one node at a time, called a *rolling upgrade*.

3.4 Maintainability

The majority of the cost of software is not in its initial development but in its ongoing maintenance. Legacy systems are especially complex to maintain. Every system we create today will one day become a legacy system (if it is valuable enough to survive for a long time), so we should design it with maintenance in mind. Three principles for maintainance are

- *operability*; the ease of keeping the system running smoothly. Automation is essential in large-scale systems, but there will always be edge cases that require manual intervention from the operations team, and since the cases that cannot be handled automatically tend to be the most complex, greater automation requires a more skilled operations team that can resolve those issues.
- *simplicity*; the ease for new engineers to understand the system. Simplicity is often subjective, as there is no objective standard of simplicity. Complexity can be broken into *essential complexity* that is inherent in the problem domain, and *accidental complexity* that arises only because of limitations of our tooling. Boundaries between the essential and the accidental shift as our tooling evolves. One of the best tools for managing complexity is *abstraction*, which can hide a great deal of implementation detail behind a clean, simple-to-understand façade, and allows for reusing similar things. Abstractions for application code can be created using methodologies such as *design patterns* and *domain-driven design* (DDD), but those are out-of-scope.
- *evolvability*; the ease of making changes to and adapting the system as requirements change. It's highly correlated with simplicity.

4 Data models

An application is modeled in terms of objects/data structures and APIs that manipulate those data structures. In order to store those data structures, you express them in terms of a general-purpose data model, e.g. JSON, XML, tables in a relational database, or vertices and edges in a graph. Document databases and relational databases started out as very different approaches to data management, but they have grown more similar over time. Relational–document hybrids are a powerful combination.

The following gives a comparison between the relational and the document data models:

	Relational model	**Document model**	**Graph model**
Data organization	Tables with rows	JSON documents (trees)	Vertices connected by edges
Schema approach	Schema-on-write: explicit schema enforced by the database on write (akin to static type checking)	Schema-on-read: implicit schema interpreted by application code on read (akin to dynamic type checking)	No schema, so no restriction on node associations
Normalization	Leans toward normalization; strong support for joins	Leans toward denormalization; weak or no support for joins in many implementations	
Relationships	One-to-many (foreign keys) and simple many-to-many (join tables) are natural	One-to-many are natural via nested lists; many-to-many relationships are awkward	Complex many-to-many relationships are natural. High-degree relationships need a "join" vertex
Data locality	Data spread across multiple tables; requires joins to reassemble	Document stored as a single continuous string; all relevant information in one place	
Impedance mismatch	Requires ORM translation layer between application objects and tablesrowscolumns	Reduced mismatch; JSON maps more closely to object structures in application code	
Referencing nested items	Can refer to any item directly by its ID	Can only reference nested items by index within the document	
Schema migration	`ALTER TABLE` and `UPDATE`; can be slow on large tables since rows may need rewriting	Write new documents with new structure; application code must handle old document formats, which is not ideal	
Readwrite tradeoff	Normalized: faster writes (single copy), slower reads (joins required)	Denormalized: faster reads (fewer joins), more expensive writes (more copies to update)	
Best suited for	Regular, structured data; many-to-many relationships; OLTP systems; data warehouses	Document-like (tree) structures (i.e. mostly one-to-many relationships); heterogeneous data; applications that load entire documents at once	Heterogenoeous data
Query language	SQL (standardized)	Varied: MongoDB aggregation pipeline, XQueryXPath (XML), JSONPath, etc.	Cypher, SPARQL, Datalog, GraphQL, SQL support for graphs
Updates	Can update individual fields and columns	Entire document usually needs to be rewritten; keep documents small and avoid frequent small updates	

4.1 Relational models

The most well known data model is the *relational* (or SQL) model, proposed by Edgar Codd in 1970, in which data is organized into *relations* (called *tables* in SQL), where each relation is an unordered collection of *tuples* (*rows* in SQL). By the mid-1980s, *relational database management systems* (RDBMSs) and SQL had become the tools of choice for most people who needed to store and query data with some kind of regular structure. Since then, there have been many competing approaches that generated a lot of hype in their time, but none lasted and the relational model always won.

Normalization is the process of structuring a relational database in accordance with a series of so-called normal forms in order to reduce data redundancy and improve data integrity. For example, instead of storing a string `region` per user/shop/whatever, you store a `regionId` that refers to a table `regions`. The string information is stored in only one place. When you store the text directly, you are duplicating the information in every record that uses it; this representation is *denormalized*. You may need to update the string information at some point, but never the ID, because the ID isn't meaningful information to humans. The downside of a normalized representation is that every time you want to display a record containing an ID, you have to do an additional lookup to resolve the ID into something human-readable using a *join*, a process called *hydrating* the IDs. Generally, normalized data is usually faster to write (since there is only one copy) but slower to read (since it requires queries with joins), whereas denormalized data is usually faster to read (fewer joins) but more expensive to write (more copies to update, more disk space used). You could view denormalization as a form of derived data. Normalized data tends to be better for OLTP systems, denormalized data for analytical systems (because they perform updates in bulk and predominantly use read-only queries). Normalized data tends to be better in systems of small to moderate scale, denormalized data in large-scale systems (because the cost of joins can be problematic). Whether to normalize or denormalize is rarely immediately obvious; the most scalable approach may involve denormalizing some things and leaving others normalized. You will have to carefully consider how often the information changes and the cost of reads and writes.

A *one-to-many relationship* in a relational database can be modeled with a foreign key column in the "many" table that references the primary key of the "one" table. A *many-to-many* relationship is usually represented as an *associative table* aka *join table*, which works well for simple cases of many-to-many relationships, but for more complex connections you

should consider a graph.

Data warehouses are usually relational, and there are a couple of widely used conventions for the structure of tables in a data warehouse that are optimized for the needs of business analysts:

- A *star schema*. At the center of the schema is a so-called *fact table*. Each row represents an event that occurred at a particular time. Each columns can either be a regular attribute, or a foreign-key reference to another table, called a *dimension table*. As each row in the fact table represents an event (e.g. a sale), the dimensions represent the who, what (e.g. the product that was sold), where, when, how, and why of the event. Queries often involve multiple joins to multiple dimension tables. A star schema consists mostly of many-to-one relationships, and other relationship types are often denormalized to simplify queries (e.g. a sale can consist of multiple products, but each purchased product is still listed as a separate event).
- A *snowflake schema*, a variation of the star schema in which dimensions are further broken into subdimensions. Snowflake schemas are more normalized than star schemas, but star schemas are often simpler for analysts to work with.
- *One big table* (OBT) take denormalization to the extreme and leave out the dimension tables entirely, folding the information in the dimensions into denormalized columns in the fact table instead (essentially, precomputing the join between the fact table and the dimension tables). Requires more storage space, but can enables faster queries. Such denormalization is unproblematic, since the data warehouse data typically represents a log of historical data that is not going to change, so you don't have the issues of data consistency and write overheads that occur with denormalization in OLTP systems.

If an application is developed in an object-oriented programming language and its data is stored in relational tables, an awkward translation layer is required between the classes, objects, and fields in the application code and the tables, rows, and columns in the database. The disconnect between the models is sometimes called an *impedance mismatch*.[7] Object-relational mapping (ORM) frameworks like ActiveRecord and Hibernate reduce the amount of boilerplate code required for this translation layer, but they are often criticized for their complexity (because they can't completely hide the differences), because some ORMs automatically generate relational schemas which can be awkward and inefficient, and because they make it easy to accidentally write inefficient queries.[8] On the other hand, ORMs reduce the amount of boilerplate code required for the translation between the persistent relational and in-memory object representation, may help with caching the results of database queries (reducing load on the database), and can help with managing schema migrations and other administrative activities.

When you want to change the format of the data, you would typically perform a *migration*. In most relational databases, adding a column with a default value is fast and unproblematic, but running the UPDATE statement or changing the datatype of a column is likely to be slow on a large table since every row needs to be rewritten or the entire table to be copied.

4.2 Document models

In the 2010s, *NoSQL* was the latest buzzword that tried to overthrow the dominance of relational databases. NoSQL doesn't refers to a single technology, but a loose set of ideas around new data models, schema flexibility, scalability, and a move toward open source licensing models. It's principles have become widely adopted, but use of the term NoSQL has faded. One lasting effect of the NoSQL movement is the popularity of the *document model*, popularized by document databases such as MongoDB and Couchbase, usually represents data as JSON[9], which are thought to be more flexible than SQL's rigid and inflexible schemas. Most document databases do not enforce any schema on the data in documents, meaning that arbitrary keys and values can be added to a document, and when reading, clients have no guarantees as to what fields the documents may contain. They're not *schemaless* as sometimes claimed, because the code that reads the data usually assumes and interprets some kind of structure (an implicit schema). A more accurate term is *schema-on-read*, in contrast with *schema-on-write* of relational databases, where the schema is explicit and the database ensures conformity when data is written. The former is similar to dynamic (runtime) type checking, and the latter to static (compile-time) type checking.

[7] A term borrowed from electronics.

[8] An example of this last problem is the N+1 query problem. Where you would normally aggregate multiple tables using a join in one SQL query, an ORM can make N separate queries for the join, which is slower than performing the join in the database. To avoid this problem, you may need to tell the ORM to everything at the same time as fetching the comments.

[9] Most relational databases have now also added JSON support. Codd's original description of the relational model allowed something similar to JSON within a relational schema, namely that a value in a row doesn't have to be a primitive datatype, but can also be a nested relation (table), so you can have an arbitrarily nested tree structure as a value. He called it *nonsimple domains*, and the JSON and XML support that was added to SQL over 30 years later is essentially this idea.

Some developers feel that the JSON model reduces the impedance mismatch between the application code and the storage layer. For instance, in a JSON representation, you can represent one-to-many relations with nested lists, which is perhaps more natural and maps more closely to an object structure in application code than the foreign keys in a relational model. It also has better *locality* than the multi-table schema; all the relevant information is in one place, making the query both faster and simpler and avoiding multiple queries or messy multiway joins. A document model is suitable if the data in your application has a document-like structure (i.e. a tree of one-to-many relationships, where typically the entire tree is loaded at once), or if the data is heterogeneous (items don't have the same structure). The document model also supports applications well in which the user can choose the order of items. A document model also has *storage locality*[10]; a document is usually stored as a single continuous string, encoded as JSON, XML, or a binary variant thereof (such as MongoDB's BSON), which has a performance advantage if your application often needs to access the entire document (or at least a large part of it), but is wasteful if you need to access only a small part. Furthermore, on updates to a document, the entire document usually needs to be rewritten, and therefore it is generally recommended that you keep documents small and avoid frequent small updates.
Many-to-one and many-to-many relationships do not easily fit within one self-contained JSON document; they lend themselves more to a normalized representation. Many-to-many relationships often need to be queried in "both directions", which could be enabled by storing ID references on both sides, which is denormalized, since the relationship is stored in two places that could become inconsistent. You also cannot refer directly to a nested item within a document, only by index. If you need to reference nested items, a relational approach works better, since you can refer to any item directly by its ID.

Document databases can store both normalized and denormalized data, but lean largely towards denormalization, because the JSON data model makes it easy to store denormalized fields, and because the weak support for joins in many document databases makes normalization inconvenient. Some document databases don't support joins at all, so you have to perform them in application code.

When you want to change the format of the data, you would just start writing new documents with the new structure and have code in the application to handle old documents when those are read, but every part of your application that reads from the database now needs to deal with documents in old formats that may have been written a long time in the past.

Query languages for document databases are varied. Some allow only key-value access by primary key, while others also offer secondary indexes to query for values inside documents, and some provide rich query languages. XML databases are often queried using XQuery and XPath, which are designed to allow complex queries, including joins across multiple documents, and format their results as XML. JSON Pointer and JSONPath provide an equivalent to for JSON. MongoDB's aggregation pipeline is a query language for collections of JSON documents and is similar in expressiveness to a subset of SQL, but it uses a JSON-based syntax rather than SQL's English sentence–style syntax. It has the `\$lookup` operator for joins.

4.3 Graph-like models

The relational model can handle simple cases of many-to-many relationships, but as the connections within your data become more complex, it becomes more natural to start modeling that data as a graph consisting of *vertices* (aka *nodes*) and *edges* (also known as *relationships*). Typical examples include social graphs, web graphs, road/rail networks. Besides *homogeneous* data, an equally powerful use of graphs is to provide a consistent way of storing completely different types of objects in a single database.

Graphs can be represented in several ways

- In an *adjacency list*, each vertex stores the IDs of its neighbor vertices that are one edge away. Suitable for graph traversals.
- An *adjacency matrix* is a two-dimensional array in which each row and column corresponds to a vertex, where the value is 0 when there is no edge between the row vertex and the column vertex and 1 when there is an edge. Suitable for machine learning.

Graphs provide several different, but related, models for structuring and querying data. We discuss the following two, which are fairly similar in what they can express, and some graph databases (such as Amazon Neptune) support both:

- The *(labeled) property graph* model. Each vertex consists of a unique identifier, a string label to describe the type of object this vertex represents, a set of outgoing edges, a set of incoming edges, and a collection of properties

[10] Storing related data together for locality is not limited to the document model. For example, Google's Spanner database offers the same locality properties in a relational data model, by allowing the schema to declare that a table's rows should be interleaved (nested) within a parent table. Oracle allows the same thing, using a feature called multi-table index cluster tables. The wide-column data model popularized by Google's Bigtable and used, for example, in HBase and Accumulo has column families, which have a similar purpose of managing locality

(key-value pairs). Each edge consists of a unique identifier, the (tail) vertex at which the edge starts, the (head) vertex at which the edge ends, a label to describe the kind of relationship between the two vertices, and collection of properties (key-value pairs). Since each vertex stores both its incoming and its outgoing edges, you can easily traverse the graph. Using different labels for different kinds of vertices and relationships enables storing several kinds of information in a single graph. You can think of a graph store as consisting of two relational tables, one for vertices and one for edges, with indices on `edges.tail_vertex` and `edges.head_vertex` to speed up reads of those. A limitation of graph models is that an edge can associate only two vertices, whereas a relational join table can represent higher-degree relationships by having multiple foreign-key references on a single row. Such relationships can be represented in a graph by creating an additional vertex corresponding to each row of the join table and edges to/from that vertex, or by using a *hypergraph*.

Cypher is a query language for property graphs, originally created for the Neo4j property graph database, later developed into an open standard as openCypher, and supported by Memgraph, KùzuDB, Amazon Neptune, Apache AGE (with storage in PostgreSQL), and others. Other graph query languages include TigerGraph's GSQL and the Property Graph Query Language (PGQL). The Graph Query Language (GQL) ISO standard, which is based on Cypher, was published in 2024, but it is not widely adopted yet.

- The *triple store* model is mostly equivalent to the property graph model, using different words to describe the same ideas. All information is stored in the form of very simple three-part statements: (*subject*, *predicate*, *object*), e.g. (Jim, likes, bananas).[11] The subject is equivalent to a vertex. The predicate and object are either a key and value pair of primitive datatypes describing a property of the subject, or an edge and head vertex originating from the subject as tail vertex.

  SPARQL is a query language for triple stores using the *Resource Description Framework* (RDF) data model. It predates Cypher, and since Cypher's pattern matching is borrowed from SPARQL, they look quite similar.
  Datalog is an even older language than SPARQL or Cypher and arose from academic research in the 1980s. It is less well-known and not widely supported in mainstream databases, but it ought to be, since it is a very expressive language that is especially powerful for complex queries. Several niche databases, including Datomic, LogicBlox, CozoDB, and LinkedIn's LIquid use Datalog as their query language. It's based on a relational data model, not a graph, but recursive queries on graphs are a particular strength of Datalog. It is a subset of Prolog.
  GraphQL is a query language that is intended for OLTP queries; its purpose is to allow client software running on a user's device to request a JSON document with a particular structure, containing the fields necessary for rendering its UI. GraphQL interfaces allow developers to rapidly change queries in client code without changing server-side APIs. That flexibility comes at a cost, however. Organizations that adopt GraphQL often need tooling to convert the queries into requests to internal services, which commonly use REST or gRPC. The language is also intentionally restricted, because GraphQL queries come from untrusted sources, and doesn't allow anything that could be expensive to execute (because of risk of DOS), recursive queries (unlike Cypher, SPARQL, SQL, or Datalog), and arbitrary search conditions. GraphQL can be implemented on top of any type of database—relational, document, or graph.

4.4 Event sourcing

It can be difficult to find a single data representation that is able to satisfy all the ways that the data needs to be queried and presented. In such situations, you can write data in one form and then derive from it representations that are optimized for different types of reads. Perhaps the simplest, fastest, and most expressive way of writing data is an *event log*, in which *events* are immutable, self-contained strings (e.g. JSONs) with a timestamp. A request from a user is called a *command*, and after it comes in it is first validated, then executed. Then it becomes a *fact* and is appended to the event log, after which several *materialized views* (aka *projections* or *read models*) are also updated. Those views should process the events in exactly the same order as they appear in the lo, which is a challenge in distributed systems. The idea of using events as the source of truth and expressing every state change as an event is known as *event sourcing*. It is similar to a star schema fact table, but rows in a fact table all have the same set of columns whereas there may be many event types with different properties in event sourcing, and a fact table is an unordered collection, while in event sourcing the order of events is important. The principle of maintaining separate read-optimized representations and deriving them from the write-optimized representation is called *command query responsibility segregation* (CQRS). It is recommended that you name your events in the past tense. Event sourcing and CQRS have several advantages: events better communicate the intent of why something happened to the developers of the system, you can always (re)derive materialized views from the event log, multiple materialized views for different read queries, building new materialized views is easy, you can write a subsequent deletion event to reverse previous erroneous events, the event log can also serve as an audit log, and

[11] Databases that offer such a data model often need to store additional metadata on each tuple, e.g. AWS Neptune uses quads (4-tuples) by adding a graph ID, and Datomic uses 5-tuples and adds a transaction ID and a Boolean to indicate deletion.

event logs can typically handle higher write throughput than databases because of their sequential access patterns (and materialized views can catch up on their own pace). Downsides are: you need to be careful with external information (e.g. exchange rates) that can jeopardize the determinism of materialization (but you could include it in events), and events being immutable can be problematic with regards to privacy compliancy if events contain personal data. EventStoreDB, MartenDB (based on PostgreSQL), and Axon Framework are specifically designed for event sourcing.

4.5 DataFrames, matrices, and arrays

DataFrames are a popular tool for data scientists preparing data for training ML models, but they are also widely used for data exploration, statistical data analysis, data visualization, and similar purposes. They're supported by R, the Pandas library for Python, Apache Spark, ArcticDB, Dask, and other systems. A DataFrame is similar to a table in a relational database or a spreadsheet and supports relational-like operators that perform bulk operations on its contents, e.g. filtering, grouping, aggregating, or joining rows. DataFrames and many libraries are optimized for *sparse matrices*, in which the majority of cells have no value. Instead of using a declarative query language, a DataFrame is generally manipulated imperatively through commands. DataFrame APIs also offer a wide variety of operations that go far beyond what relational databases offer, and the data model is often used in ways that are very different from typical relational data modeling. A common use of DataFrames is to transform data from a relational-like representation into a matrix or multidimensional array representation (which is the form in which many ML algorithms expect their input). DataFrames are also used in the financial industry for representing time-series data.

A *matrix* can contain only numbers, and various techniques are used to transform nonnumerical data into numbers in the matrix. For example, dates could be scaled to be floating-point numbers within a suitable range. Or, columns that can take only one of a small, fixed set of values (enums) can be *one-hot encoded*, meaning that for each possible value a column in the matrix is created, with a 1 indicating the value for a particular row.

5 Case studies

5.1 Twitter

Let's assume that users make a total of 500 million posts per day (5,800 posts per second) on average, but that the rate can spike to as high as 150,000 posts per second. Let's also assume that the average user follows 200 people and has 200 followers, but this is highly variable. The data is kept in a relational database, with tables for `users`, `posts`, and `follows` (which links users IDs to user IDs); a normalized representation. The main read operation that our social network must support is the home timeline, with recent posts by people the user is following. The following SQL could support that:

```
SELECT posts.*, users.*
FROM posts
JOIN follows ON posts.sender_id = follows.followee_id
JOIN users ON posts.sender_id = users.id
WHERE follows.follower_id = current_user
ORDER BY posts.timestamp DESC
LIMIT 1000
```

After somebody makes a post, we want their followers to be able to see it within five seconds. The user's client could query every five seconds (this is known as *polling*), but if we assume 10 million users are logged in, that would be 2 million queries per second, which is a lot. This query is also quite expensive; the query needs to fetch a list of recent posts by ≈ 200 people and merge those lists, and that's only the average case.

We can do better than polling. For each user, we store a data structure containing their home timeline (the recent posts by followees). Every time a user makes a post, we look up all their followers and insert that post into the home timeline of each follower. In other words, a post creation request *fans-out* in ≈ 200 downstream requests. We now need to do more work every time a user makes a post; at a rate of 5,800 posts per second, we do just over 1 million home timeline writes per second, which is a lot, but significantly less than 400 million per-sender post lookups per second. If the rate of posts spikes (because of a special event), we can enqueue them and accept that it will temporarily take longer for posts to show up, but timelines remain fast to load, since we serve them from a cache. This process of precomputing and updating the results of a query is called *materialization*, and the timeline cache is an example of a materialized view. This speeds up reads, but writes are more work.

Two considerations. First, if a user is following a very large number of accounts and those accounts post a lot, that user will have a high rate of writes to their materialized timeline. However, that user is not likely reading all the posts in their timeline, so we can drop some of their timeline writes. Second, when a celebrity account makes a post, we have to insert that post into the home timelines of their millions of followers. Dropping some of those writes is not OK. One way

of solving this problem is to handle celebrity posts separately from pleb posts by storing celebrity posts separately and merging them with the materialized timeline when it is read. Despite such optimizations, handling celebrities on a social network can require a lot of infrastructure.

The home timeline would be considered a denormalized representation (and derived data). However, don't want to store the post text. Instead, we normalize again by storing a post ID together with the poster's user ID, because the data they refer to (post text, like count, repost count, username, user profile picture) can and will be updated, and the storage cost would be increased significantly by such denormalization. This does mean that whenever the timeline is read, the service still needs to perform two joins/hydrations: it fetches the post content and statistics by post ID and the sender's profile by user ID, but this is actually a fairly easy operation to scale since it parallelizes well, and the cost doesn't depend on the number of followees or followers.